

COSC2129/1476
Artificial Intelligence

Planning

Road Map for Today

- ❑ Revision of logic fundamentals and inference
- ❑ Classic planning
 - Definition and representation
 - Examples
- ❑ Planning algorithms
 - Forward and backward state-space search
 - Heuristics
 - Other planning approaches

Important references (pdf already in the blackboard):

1. **Chapter 11** of Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach* (**Second Edition**), Prentice Hall, 2002
2. **Chapter 8.4** of George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving* (Sixth Edition), Addison-Wesley, 2008

Logic Fundamentals

- ❑ Syntax and semantics
- ❑ Models and possible worlds
- ❑ Logical entailment ($KB \models \alpha$) and inference
- ❑ Equivalence, validity, and satisfiability
- ❑ Soundness and completeness of an inference algorithm

Logical Inferences Rules

Modus Ponens:

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

And-Elimination:

$$\frac{\alpha \wedge \beta}{\alpha}$$

**FOL: Lifted Version
(Unification)**

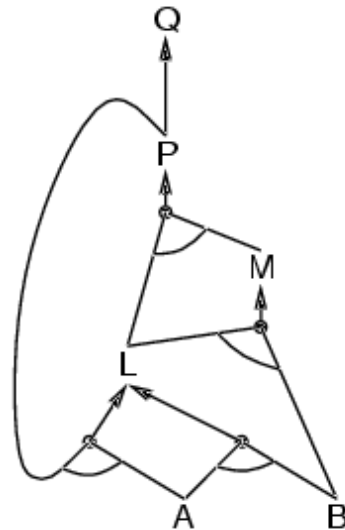
Logical equivalence

Resolution:

$$\frac{l_i \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{l_i \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

Logical Inferences Algorithms

- ❑ Model checking
- ❑ Forward chaining and backward chaining



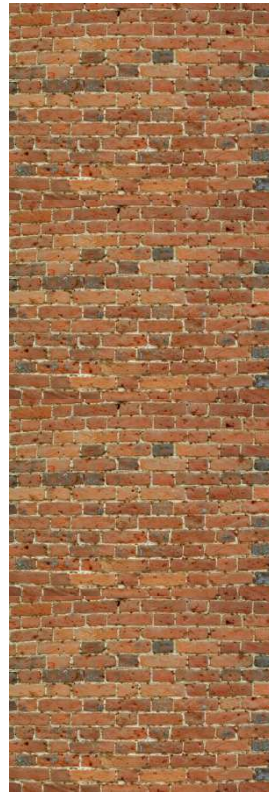
AND-OR graph

- ❑ Resolution-based theory proving

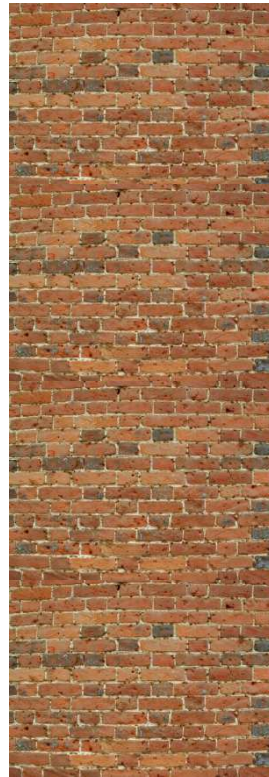
Planning

DARPA Grand Challenge

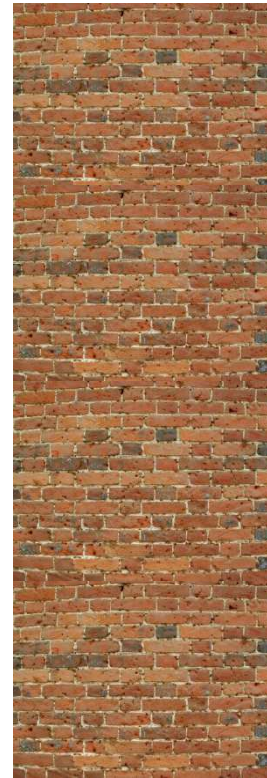
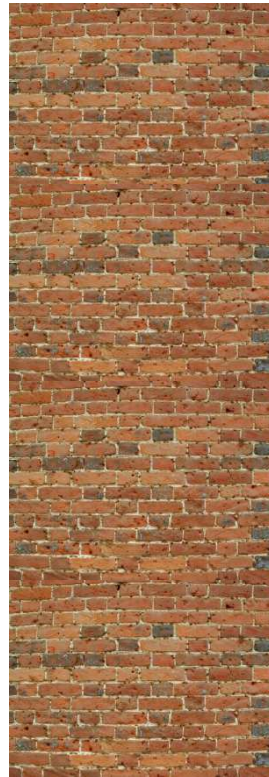
A prize competition for American autonomous vehicles, funded by the Defence Advanced Research Project Agency (DARPA)



DARPA Grand Challenge

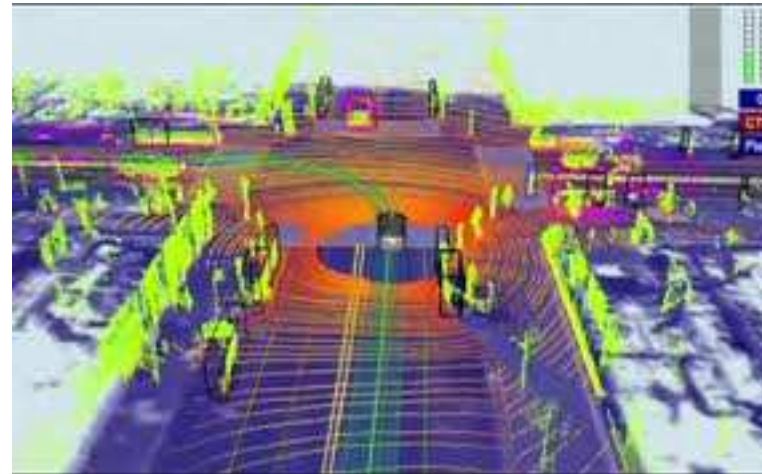


DARPA Grand Challenge



Google's Driverless Car

- ❑ Now road-legal in Nevada
 - Robot cars will be issued a red number plate.



Google's Self-Driving Car Takes Blind Man for a Ride:

<http://www.pcmag.com/article2/0,2817,2402340,00.asp>

Planning

Declarative Knowledge focuses on the description of the world (So far)

Procedural Knowledge focuses on how to change the world (now)

Need to specify how an action changes the world!

Consider a robot that performs some specific tasks as follows:

- Navigating from one room to another
- Putting together a water pump
- Safely shutting down a nuclear power plant
- Managing the operations of the space station

To accomplish these tasks, the robot needs to create **plans of actions**.

Classic planning is knowledge-intensive, focusing on fully observable, static, finite, deterministic environments involving a single agent (robot).

Plans and Planning

A plan is a sequence of actions that allows to accomplish a certain task or results in a specified goal.

An action is a primitive operation that can be carried out by an agent (robot).

Actions are usually specified not at the micro-level but at a higher level.

-- Walk (*not* lift up right foot and place in front of left foot ...)

-- Turn left (*not* turn the left stepper motor 3 revolutions)

Planning is the process of determining a sequence of actions necessary to achieve a specified goal.

Planning involves search.

Plans cannot be pre-programmed – an agent (robot) must be able to automatically derive plans for different tasks.

Plans and Planning (cont'd)

A planning system is an AI system that constructs plans for different tasks.

A general-purpose planning system can be tailored to different domains and can deal with novel situations.

Some applications:

- ❑ *Robot control*: many different planning systems for many different robots. One of the earliest systems was the STRIPS planning system, used to control the robot *SHAKY*.
- ❑ *Manufacturing*: O-PLAN have been used at Hitachi; ISIS has been used at Westinghouse.
- ❑ *Mission planning*: the Hubble space telescope uses a planner for allocating tasks to the telescope's viewing schedule.
- ❑ *Intelligent agents*: lots of very recent technologies for the control of small “intelligent” processes, used in distributed digital libraries, software processes, animated intelligent creatures in games and simulations, etc.

Planning as Search

The task of constructing a plan can be formulated as a state space search problem.

- States: initial state; goal state; successor/predecessor
- Operations: actions which change the world.
- Goal test: whether a given state is the goal state.

State space is defined by the set of all states reachable from the initial state by any sequence of actions, which forms a graph.

Constructing a plan is to answer the question: *how do I change the current world so that my goal can be achieved.*

Constructing a plan involves:

- Considering the possible actions that an agent could perform, and then
- Considering the effects of each action, and then
- Devising a sequence of actions (i.e., a plan) that will achieve the goal.

Knowledge Representation for Planning

In planning with state-space search, we need to represent:

- **State knowledge** (*Declarative knowledge*)
- **Operator knowledge** (*Procedural knowledge*)

For state knowledge, we can use logic representation:

- PL and FOL state description
- Description of a state could be complex and requires a large number of sentences
- Description of a state could be problem-dependent or problem-independent

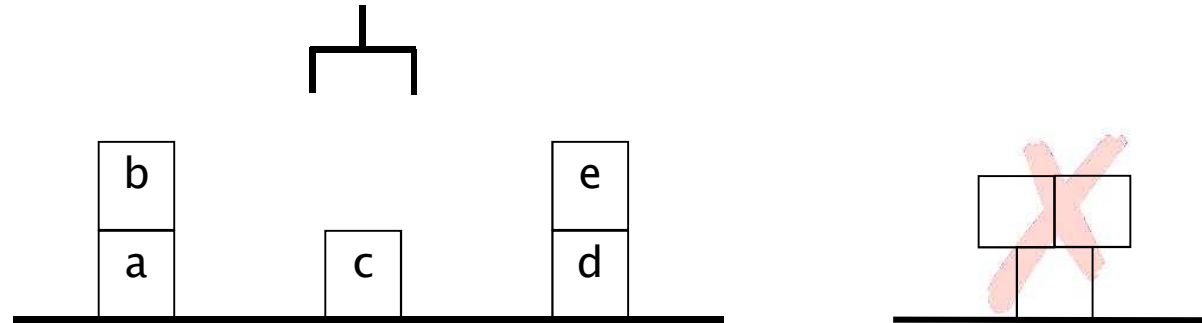
For operator knowledge, we can use:

- Logic representation
- PDDL representation, e.g., STRIPS (Planning Domain Definition Language)

Ideal planning problem representation should scale up to large and real-world problems.

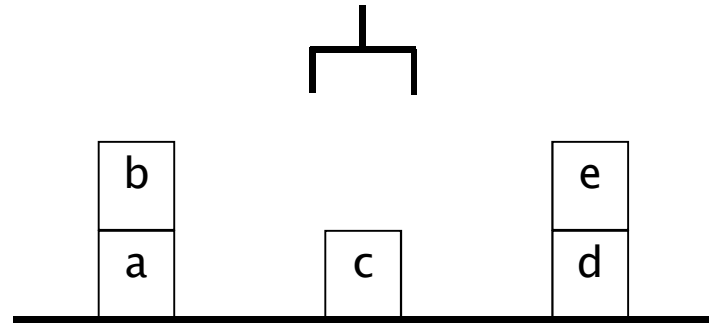
Any state-space search algorithms can be used, but heuristic search is highly preferred.

Blocks World



- ❑ One of the most famous planning domain consisting of a set of cube-shaped blocks sitting on a table.
- ❑ Blocks can be stacked but only one block can be directly on top of another.
- ❑ The robot arm has a gripper that can grasp any block with no block on top of it (i.e., clear block), and only one block can be grasped by the gripper at a time.
- ❑ A grasped block can be moved to any location on the tabletop or placed on top of any other clear block.
- ❑ The goal is usually to build one or more stacks of blocks.

Blocks World: State Representation



The state of the world is described in FOL via a set of predicates:

Location(W, X, Y, Z)

Block *W* is at coordinates *X, Y, Z*.

On(W, V)

Block *W* is immediately on top of block *V*.

Clear(W)

Block *W* has nothing on top of it.

Gripping(W)

The robot arm's gripper is holding block *W*.

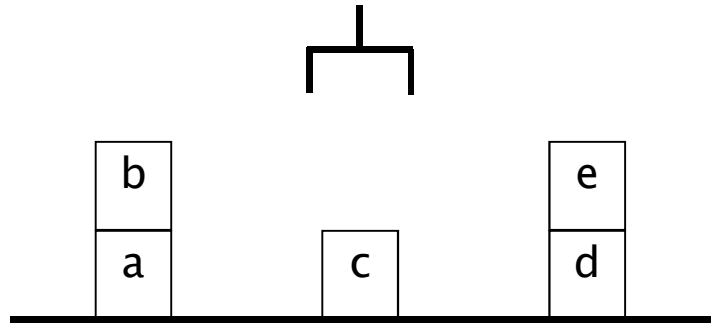
Gripping()

The robot arm's gripper is empty.

Ontable(W)

Block *W* is on the table

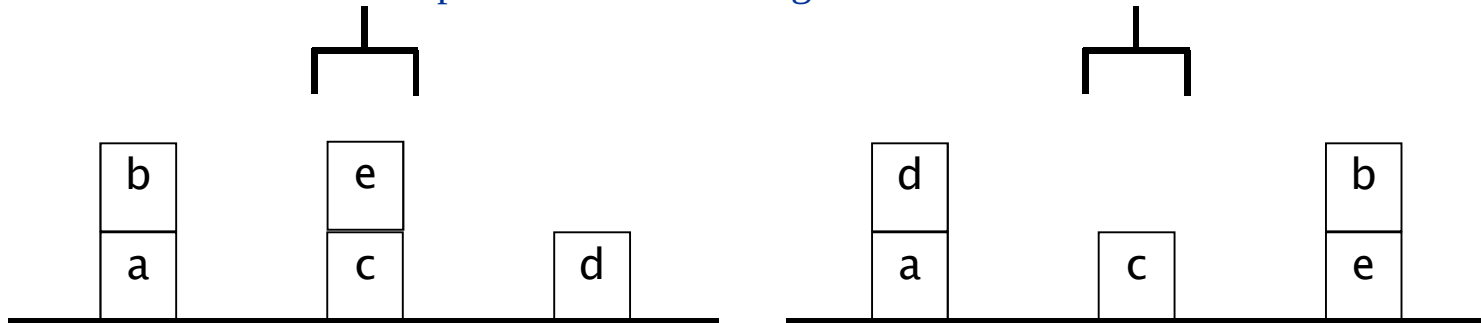
Blocks World: State Representation (cont'd)



The above state can be represented as:

$Ontable(a) \wedge OnTable(c) \wedge OnTable(d) \wedge On(b, a) \wedge On(e, d) \wedge Clear(b) \wedge Clear(c) \wedge Clear(e) \wedge Gripping()$

How about the full state description of the following two states?



Blocks World: Actions

Goto(X, Y, Z)

Go to location (X, Y, Z).

Usually implicit in *Pickup(W)*.

Pickup(W)

Pick up block *W* and hold it.

W must be clear; gripper must be empty; location of *W* is known.

Putdown(W)

Place block *W* down on table.

W must be held in gripper.

Stack(W, V)

Place block *W* on top of block *V*.

Gripper must be holding *W*; *V* must be clear.

Unstack(W, V)

Remove block *W* from block *V*.

W must be clear; *W* must be on *V*; gripper must be empty.

Description can be simplified by recording the current location of each block and keeping track of its movements to the new location. Therefore, *Goto(X, Y, Z)* is no longer necessary.

These are primitive operators.

-- It is assumed that the robot arm (gripper controller) can perform these actions

-- It is not necessary to break actions into smaller steps.

Logic Representation of Actions

Actions can be described by rules that operate on states and generate new states.

These rules has the form: $A \Rightarrow (B \Leftarrow C)$ which means that action A produces new predicate(s) B when condition(s) C is(are) true.

Pickup action:

$$\forall X (Pickup(X) \Rightarrow (Gripping(X) \Leftarrow Gripping() \wedge Clear(X)))$$

-- For all blocks X, when the gripper is empty and X is clear we can get a new state with the gripper holding X by applying action Pickup.

Putdown action:

$$\forall X (Putdown(X) \Rightarrow (Gripping() \wedge Ontable(X) \wedge Clear(X) \Leftarrow Gripping(X)))$$

-- For all blocks X, when the gripper is holding X we can get a new state with the gripper being empty, X being on the table and X being clear by applying action Putdown.

Logic Representation of Actions (cont'd)

Stack action:

$$\forall X \forall Y (Stack(X, Y) \Rightarrow (On(X, Y) \wedge Gripping() \wedge Clear(X) \Leftarrow (Gripping(X) \wedge Clear(Y))))$$

-- For all blocks X and Y, when Y is clear and the gripper is holding X we can get a new state where X is on Y, the gripper is empty, and X is clear by applying action **Stack**.

Unstack action:

$$\forall X \forall Y (Unstack(X, Y) \Rightarrow ((Gripping(X) \wedge Clear(Y)) \Leftarrow (On(X, Y) \wedge Gripping() \wedge Clear(X))))$$

-- For all blocks X and Y, when X is on Y, X is clear and the gripper is empty we can get a new state where Y is clear and the gripper is holding X by applying action **Unstack**.

We can use these rules to generate new states in a space.

Frame Axioms

Actions specify what changes. **How do we specify what doesn't change?**

Frame problem: representing all the things that stays the same.

Frame axioms are rules defined to say what predicates describing a state are not changed after an operator is applied, and thus remain in the new state.

- It is necessary to have a frame axiom for each pair of actions and predicates.
- It is tractable only for very small problems.

Frame axiom examples:

$\forall X \forall Y \forall Z (Unstack(Y, Z) \Rightarrow (Ontable(X) \Leftarrow OnTable(X)))$

- For all blocks X, Y and Z, when X is on the table we can get a new state with X still being on the table by applying action Unstack(Y, Z).

$\forall X \forall Y \forall Z (Stack(Y, Z) \Rightarrow (Ontable(X) \Leftarrow OnTable(X)))$

- Stacking Y on Z does not change the position of any block on the table.

Performing Unstack

Initial state

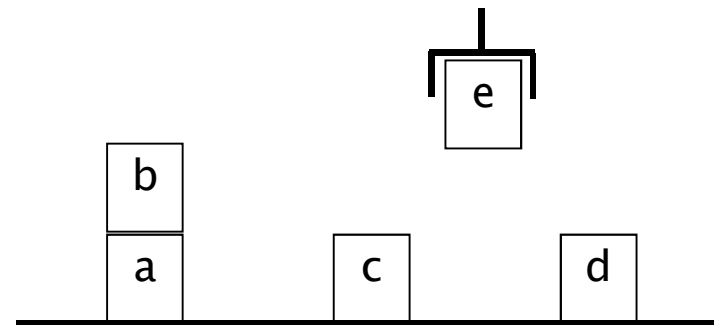
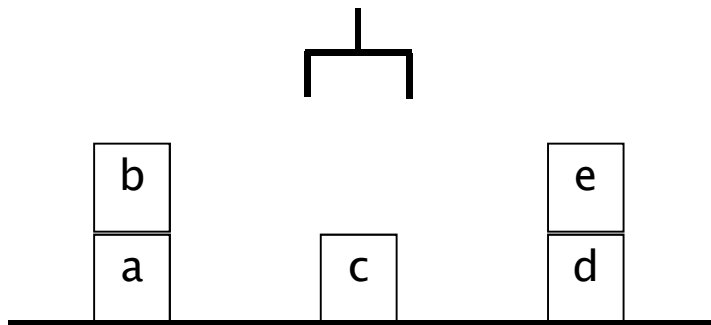
$Ontable(a) \wedge OnTable(c) \wedge OnTable(d) \wedge On(b, a) \wedge On(e, d) \wedge Clear(b)$
 $\wedge Clear(c) \wedge Clear(e) \wedge Gripping()$

Applying action Unstack(e, d)

$\forall X \forall Y (Unstack(X, Y) \Rightarrow ((Gripping(X) \wedge Clear(Y)) \Leftarrow (On(X, Y) \wedge Gripping() \wedge Clear(X))))$

New state after Unstack(e, d) is applied

$Ontable(a) \wedge OnTable(c) \wedge OnTable(d) \wedge On(b, a) \wedge Clear(b)$
 $\wedge Clear(d) \wedge Clear(c) \wedge Gripping(e)$



Issues: Planning via FOL and Search

- Using frame axioms adds exponentially to the search.
 - Frame axioms are needed for each pair of predicate and action.
 - Assumes sub-problems are independent and can be solved in any order
- (Usually not the case, in building a house the walls must be built before the roof.)

STRIPS

- ❑ **S**Tanford **R**esearch **I**nstitute **P**roblem Solver (**STRIPS**)
- ❑ Automated Planning System used to drive the SHAKEY robot of the early 1970s
- ❑ More commonly referring to the formal language of the inputs to this planner
- ❑ Base for most of the languages for expressing automated planning problems
- ❑ STRIPS addressed the following problems:
 - Efficiently representing and implementing actions of a planner (action schema)
 - Conflicting subgoals (triangle tables)

History of STRIPS: <https://www.youtube.com/watch?v=mQ7M-zhiu7U>

STRIPS Representation

- ❑ **States:** a conjunction of ground and function-free positive literals
 - $Poor \wedge Unknown$ and $At(Plane1, Melbourne) \wedge At(Plane2, Sydney)$ are legal.
 - $At(x, y)$ and $At(Father(Fred), Sydney)$ are not allowed.
 - Closed-world assumption: any conditions not mentioned are assumed false
- ❑ **Goals:** a partially specified state
- ❑ **Actions:** action schemas, composed of the **action name and parameter list**, **pre-condition**, and **effect** (add and delete lists).
 - Applicable actions (substitution may apply)
 - The results of executing an applicable action (STRIPS assumption)
- ❑ STRIPS avoids the need of frame axioms (partial known, action-centric representation).
- ❑ Solution for a planning problem: an action sequence that, when executing in the initial state, results in a state that satisfies the goal.

STRIPS Representation of Blocks World

Pickup(X) PRE: $Gripping() \wedge Clear(X) \wedge Ontable(X)$
 ADD: $Gripping(X)$
 DEL: $Gripping() \wedge Clear(X) \wedge Ontable(X)$

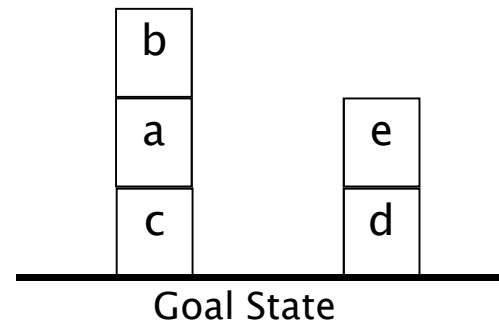
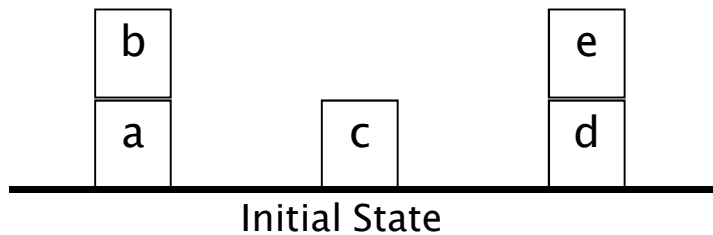
Putdown(X) PRE: $Gripping(X)$
 ADD: $Gripping() \wedge Ontable(X) \wedge Clear(X)$
 DEL: $Gripping(X)$

Stack(X, Y) PRE: $Gripping(X) \wedge Clear(Y)$
 ADD: $On(X, Y) \wedge Gripping() \wedge Clear(X)$
 DEL: $Gripping(X) \wedge Clear(Y)$

Unstack(X, Y) PRE: $Gripping() \wedge Clear(X) \wedge On(X, Y)$
 ADD: $Gripping(X) \wedge Clear(Y)$
 DEL: $Gripping() \wedge Clear(X) \wedge On(X, Y)$

Incompatible Subgoals

Suppose the task is:



In the goal we require $On(b, a) \wedge On(a, c)$

$On(b, a)$ is already true, so we might think we just have to arrange $On(a, c)$

This is impossible without first undoing $On(b, a)$ by performing action unstack.

The *triangle table* addresses such problems.

Triangle Table

- A way of re-using previously developed plans
- Store “Macro Actions” such as $Stack(X, Y) \wedge Stack(Y, Z)$
- Fits initial state: $X = b, Y = a, Z = c$
- Macro action is interchangeable with normal action

- Atomic actions are recorded along the diagonal
- Preconditions in rows
- Postconditions in columns
- Can be used in recovering from accidents
 - Find which kernel is true
 - Continue with the next action

1	gripping() clear(X) on(X,Y)	unstack(X,Y)					
2		gripping(X)	putdown(X)				
3	ontable(Y)	clear(Y)	gripping()	pickup(Y)			
4	clear(Z)			gripping(Y)	stack(Y,Z)		
5			clear(X) ontable(X)		gripping()	pickup(X)	
6					clear(Y)	gripping(X)	stack(X,Y)
7					on(Y,Z)		on(X,Y) clear(X) gripping()
	1	2	3	4	5	6	7

Planning Algorithms

- ❑ Planning inherently maps into a state-space search problem.
- ❑ Planning problems can be solved with any search algorithms while keeping track of actions used to reach the goal.
- ❑ Declarative representation of action schemas (in STRIPS language) enables both forward search from the initial state and the backward search from the goal.
- ❑ Planning with forward state-space search – progression planning
 - irrelevant action problem
 - large state space
- ❑ Planning with backward state-space search – regression planning
 - relevant and consistent actions
- ❑ Good heuristic functions enables practical feasibility.

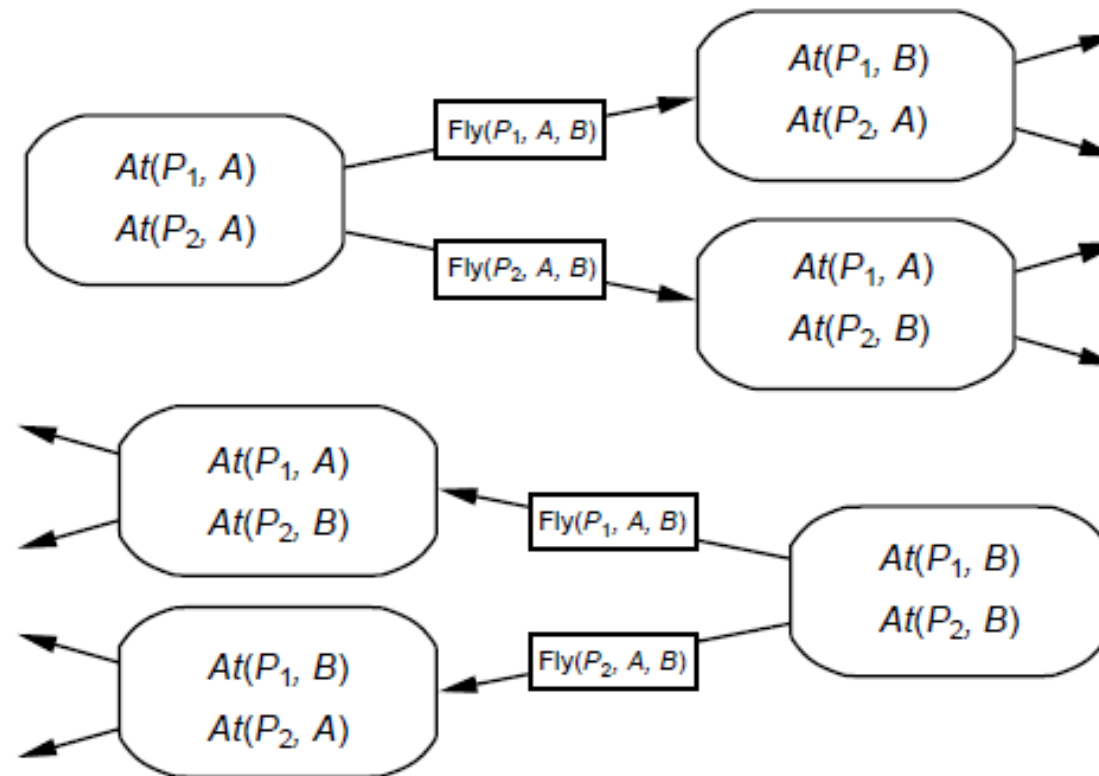
Forward State-Space Search

- ❑ Starting in the problem's initial state, and considering sequences of actions until a sequence that reaches the goal state is found.
- ❑ **Initial state:** a set of possible literals (not appearing ones are false)
- ❑ **Actions applicable to a state:** preconditions being satisfied by this state
- ❑ **The successor state** resulting from an action: adding/deleting positive/negative effect literal; applying the unifier from the preconditions to the effect literals
- ❑ **Goal test:** whether a state satisfies the goal
- ❑ Step cost of each action: 1
- ❑ Too many irrelevant actions and too large state space
- ❑ Must use good heuristic functions to enable practical feasibility

Backward State-Space Search

- ❑ Starting at the goal, and applying the actions backwards until we find a sequence of steps that reaches the initial state.
- ❑ In any state, only considering actions relevant and consistent to that state, which results in a set of predecessor states
 - **relevant** action: if achieving any of the conjuncts of the goal
 - **consistent** action: if not negating any of the conjuncts of the goal
- ❑ Works only when we know how to regress from a state description to the predecessor state description (STRIPS description makes this easy)
- ❑ Given a **state description** and a **relevant and consistent action**, the predecessor state is constructed by **deleting** the action's positive effect literals from and **adding** each of the action's precondition literals (unless they already appear) to the given state description.

Forward and Backward Search Example



Heuristics

- ❑ Neither forward nor backward search is efficient without a good heuristic function.
- ❑ Heuristic functions estimate the distance from a state to the goal.
- ❑ Basic design principle: looking at the effects of the actions and the goal to be achieved and then guessing how many actions needed to achieve the goal.
- ❑ Defining a relaxed problem and/or considering the subgoal independence assumption.
- ❑ Heuristic examples:
 - *ignore preconditions heuristic*: number of unsatisfied subgoals
 - *minimum set-cover heuristic*: minimum #actions with the union of positive effects satisfying the goal
 - *ignore delete lists (empty-delete-list) heuristic*: removing negative effects while retaining preconditions

Demo: STRIPS Planning

An applet that demonstrates planning using the blocks world problem domain and STRIPS representation:

<http://www.aispace.org/planning/index.shtml>

Other Planning Approaches

- ❑ Planning graphs
- ❑ Classic planning as Boolean satisfaction
- ❑ Planning as first-order logic deduction
- ❑ Planning as constraint satisfaction
- ❑ Planning as refinement of partially ordered plans

Challenges in Planning

- Re-planning if a plan unexpectedly fails
- Reasoning about actions that consume time
- Resources that are shared between different processes
- Planning to meet deadlines
- Uncertainty in the world
- Planning in a dynamic and unpredictable world
- Planning by co-operating agents
- Planning is not a solved problem, still lots of research in progress

Conclusion

- ❑ Revision of logic fundamentals and inference
- ❑ Classic planning
 - Definition and representation
 - Examples
- ❑ Planning algorithms
 - Forward and backward state-space search
 - Heuristics
 - Other planning approaches